

■ Spring Boot Actuator

Complete Endpoint Reference — Interview Notes

What is Spring Boot Actuator?

Actuator is a Spring Boot sub-project that adds **production-ready monitoring and management** features to your application. It exposes HTTP endpoints that tell you everything about your running app — health, metrics, config, beans, mappings — without writing a single line of monitoring code.

Add dependency (that's all you need):

```
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId> </dependency>
```

Expose all endpoints in application.properties:

```
# Expose all endpoints (default: only /health is exposed)
management.endpoints.web.exposure.include=* # Show full health details always
management.endpoint.health.show-details=always # Change actuator base path (optional, default
is /actuator) management.endpoints.web.base-path=/actuator
```

■■ SECURITY WARNING: Never expose all endpoints (*) in production without Spring Security. /actuator/env exposes passwords, /actuator/beans exposes internals. Always restrict in prod.

All Endpoints — Quick Reference

Endpoint	URL	Default Exposed?	Use When
health	/actuator/health	■ Yes	Check if app is UP/DOWN
info	/actuator/info	■ Yes	Show app name, version, build info
metrics	/actuator/metrics	■ No	CPU, memory, HTTP request stats
beans	/actuator/beans	■ No	Debug which beans are loaded
env	/actuator/env	■ No	See all config properties
mappings	/actuator/mappings	■ No	See all registered API endpoints
loggers	/actuator/loggers	■ No	View/change log levels at runtime
httptrace	/actuator/httptrace	■ No	Last 100 HTTP requests trace
threaddump	/actuator/threaddump	■ No	Debug thread states, deadlocks
heapdump	/actuator/heapdump	■ No	Download JVM heap dump file
shutdown	/actuator/shutdown	■ No	Gracefully shut down app (POST)

1. /actuator/health

Shows whether your application and all its components (DB, disk, Kafka, Redis etc.) are UP or DOWN. This is the most important endpoint — used by Kubernetes liveness/readiness probes, load balancers, and monitoring tools to know if the app should receive traffic.

Configuration:

```
# Show full component details management.endpoint.health.show-details=always # always show details
management.endpoint.health.show-details=never # only UP/DOWN (default)
management.endpoint.health.show-details=when-authorized # only for authenticated users
```

Sample Response (show-details=always):

```
{ "status": "UP", "components": { "db": { "status": "UP", "details": { "database": "MySQL",
"validationQuery": "isValid()" } }, "diskSpace": { "status": "UP", "details": { "total":
499963174912, "free": 200000000000 } }, "ping": { "status": "UP" } } }
```

Status	Meaning	HTTP Code
UP	All components healthy	200 OK
DOWN	One or more components failed	503 Service Unavailable
OUT_OF_SERVICE	Manually taken offline	503
UNKNOWN	Cannot determine health	200

When to use /health:

- Kubernetes liveness probe — is app alive? Restart if DOWN
- Kubernetes readiness probe — is app ready for traffic?
- Load balancer health check — remove from rotation if DOWN
- Uptime monitoring tools (Grafana, Prometheus alerts)

Custom HealthIndicator:

```
@Component public class PaymentServiceHealthIndicator implements HealthIndicator { @Override
public Health health() { boolean isUp = checkExternalPaymentService(); // your logic if (isUp)
return Health.up() .withDetail("message", "Payment API reachable") .withDetail("responseTime",
"120ms") .build(); else return Health.down() .withDetail("message", "Payment API unreachable")
.withDetail("error", "Connection timeout") .build(); } }
```

- Now /actuator/health will show 'paymentService' as a component automatically.

■ Interview Q: How does Actuator know if DB is UP or DOWN?

Spring Boot has built-in HealthIndicators for common components. DataSourceHealthIndicator runs a test query against DB — if it succeeds, status is UP. If DB is unreachable, DOWN. You can write custom HealthIndicator for any third-party service.

■ Interview Q: When should show-details be 'always' vs 'when-authorized'?

In development: always — easier debugging. In production: when-authorized — health details can reveal DB type, disk paths, internal config which is a security risk. Use when-authorized so only authenticated admins see full details.

2. /actuator/info

Shows application metadata — name, version, build time, git commit info. By default returns empty {}. You configure what to show. Used in deployments to verify which version of the app is actually running.

Configuration:

```
# application.properties info.app.name=eCommerce Application info.app.version=1.0.0
info.app.description=Spring Boot Microservices eCommerce Project info.app.developer=Your Name
info.java.version=21 # Enable env info contributor management.info.env.enabled=true
```

Sample Response:

```
{ "app": { "name": "eCommerce Application", "version": "1.0.0", "description": "Spring Boot
Microservices eCommerce Project", "developer": "Your Name" }, "java": { "version": "21" } }
```

When to use /info:

- Verify correct version is deployed after release
- Show build metadata in dashboards
- Track which git commit is running in production
- Never expose sensitive internal info here — it's public by default

■ Interview Q: /actuator/info returns empty {} — why?

Because info endpoint only shows what you explicitly configure via info.* properties or InfoContributors. By default nothing is configured so it returns empty. You must add management.info.env.enabled=true and info.* properties to populate it.

3. /actuator/metrics

Exposes JVM, system, and application metrics. This is the backbone of Prometheus + Grafana integration. Shows CPU usage, memory, HTTP request counts, response times, DB connection pool stats etc.

Hit /actuator/metrics to list all available metrics:

```
{ "names": [ "jvm.memory.used", "jvm.memory.max", "jvm.gc.pause", "jvm.threads.live",  
"http.server.requests", "hikaricp.connections.active", "hikaricp.connections.pending",  
"process.cpu.usage", "system.cpu.usage", "tomcat.sessions.active.current" ] }
```

Drill into a specific metric:

```
# GET /actuator/metrics/http.server.requests { "name": "http.server.requests", "measurements":  
[ { "statistic": "COUNT", "value": 1523.0 }, { "statistic": "TOTAL_TIME", "value": 45.231 }, {  
"statistic": "MAX", "value": 2.105 } ], "availableTags": [ { "tag": "method", "values": ["GET",  
"POST", "PUT"] }, { "tag": "status", "values": ["200", "404", "500"] }, { "tag": "uri",  
"values": ["/api/products", "/api/users"] } ] }
```

Metric	What it tells you	Alert when
jvm.memory.used	Heap memory used by JVM	Approaches jvm.memory.max
process.cpu.usage	CPU % used by your app	Stays above 80% consistently
http.server.requests	Count, avg time, max time per API	Max response time spikes
hikaricp.connections.active	DB connections currently in use	Equals pool max size
hikaricp.connections.pending	Threads waiting for DB connection	Greater than 0 frequently
jvm.threads.live	Active JVM threads	Keeps growing (thread leak)
jvm.gc.pause	Time spent in garbage collection	Long GC pauses (> 1s)

Custom metric example:

```
@Service @RequiredArgsConstructor public class OrderService { private final MeterRegistry  
meterRegistry; // inject Micrometer registry public Order placeOrder(Order order) { // your  
logic meterRegistry.counter("orders.placed", "status", "success").increment(); return  
orderRepo.save(order); } } // Now /actuator/metrics/orders.placed shows order count!
```

When to use /metrics:

- Feed into Prometheus for time-series data
- Visualize in Grafana dashboards
- Set alerts — e.g. alert if CPU > 80% or DB connections full
- Do NOT expose to public — reveals internal performance data

■ Interview Q: How do Actuator metrics connect to Grafana?

Add micrometer-registry-prometheus dependency → Actuator exposes /actuator/prometheus endpoint in Prometheus format → Prometheus scrapes it every 15s → Grafana queries Prometheus → dashboards and alerts. Actuator is the data source, Prometheus is the collector, Grafana is the visualizer.

4. /actuator/beans

Lists ALL Spring beans currently loaded in the ApplicationContext — their type, scope, dependencies, and which class defined them. Purely a debugging/development tool.

Sample Response (trimmed):

```
{ "contexts": { "application": { "beans": { "userServiceCls": { "aliases": [], "scope": "singleton", "type": "com.ecom.application.Service.UserService.UserServiceCls", "resource": "file [...UserServiceCls.class]", "dependencies": ["userRepository"] }, "productController": { "scope": "singleton", "type": "com.ecom.application.Controller.ProductController", "dependencies": ["productService"] } } } } }
```

When to use /beans:

- Debug why a bean is not getting injected — check if it's loaded at all
- Verify @Component/@Service/@Repository are picked up by component scan
- Check bean dependencies — see what each bean depends on
- NEVER expose in production — reveals full internal architecture
- Only useful in development/debugging — disable in prod

■ Interview Q: When would you use /actuator/beans?

When a @Autowired injection fails or a bean is unexpectedly null. Check /beans to verify the bean is registered at all. If it's missing, your @Component or @Service annotation is not being picked up — usually a package scan issue.

5. /actuator/env

Shows ALL environment properties — application.properties values, system environment variables, JVM system properties, command-line arguments. Extremely powerful for debugging config issues but extremely dangerous to expose publicly.

Sample Response (trimmed):

```
{ "activeProfiles": ["dev"], "propertySources": [ { "name": "applicationConfig:
[classpath:/application.properties]", "properties": { "spring.datasource.url": { "value":
"jdbc:mysql://localhost:3306/ecom_db" }, "spring.datasource.password": { "value": "*****" //
masked by default }, "server.port": { "value": "8080" } } }, { "name": "systemEnvironment",
"properties": { "JAVA_HOME": { "value": "/usr/lib/jvm/java-21" } } } ] }
```

Drill into specific property:

```
# GET /actuator/env/server.port { "property": { "source": "application.properties", "value":
"8080" }, "activeProfiles": ["dev"] }
```

When to use /env:

- Debug config — verify correct property value is loaded
- Check which profile is active (dev/prod/staging)
- Verify environment variables are being picked up
- HIGHEST RISK endpoint — can expose DB passwords, API keys, secrets
- In production: exclude this endpoint completely

```
# Exclude specific endpoints in production
management.endpoints.web.exposure.exclude=env,beans,heapdump
```

■ Interview Q: Is /actuator/env safe to expose in production?

No. Even though Spring masks some sensitive values like passwords, it can still expose DB URLs, API keys, internal hostnames, and config structure. In production either exclude it completely or protect with Spring Security so only ADMIN role can access it.

6. /actuator/mappings

Lists all HTTP request mappings registered in your application — every `@GetMapping`, `@PostMapping` etc. with their handler class and method. Great for documentation and debugging.

Sample Response (trimmed):

```
{ "contexts": { "application": { "mappings": { "dispatcherServlets": { "dispatcherServlet": [ {
  "handler": "com.ecom.Controller.ProductController#getProduct(Long)", "predicate": "{GET
[/api/products/{id}]}", "details": { "requestMappingConditions": { "methods": ["GET"],
"patterns": ["/api/products/{id}"] } } }, { "handler":
"com.ecom.Controller.UserController#saveUser(User)", "predicate": "{POST
[/api/users/saveUser/]}" } ] } } } }
```

When to use /mappings:

- Verify your `@GetMapping` URL is registered correctly

- Debug 404 errors — endpoint not found issues

- Generate API documentation base

- Exposes full API surface — exclude in production

■ Interview Q: Getting 404 on your endpoint — how do you debug?

Hit `/actuator/mappings` and check if your endpoint URL is listed. If missing, the `@RequestMapping` or `@GetMapping` URL is wrong, or `@RestController` is missing. If listed, the URL you're calling doesn't match — check path variables, request method, or base path.

7. /actuator/loggers

Shows current log levels for all loggers and — most powerfully — lets you CHANGE log levels at runtime without restarting the app. This is invaluable in production debugging.

GET /actuator/loggers — list all loggers:

```
{ "levels": ["TRACE", "DEBUG", "INFO", "WARN", "ERROR", "OFF"], "loggers": { "ROOT": {  
  "configuredLevel": "INFO", "effectiveLevel": "INFO" }, "com.ecom": { "configuredLevel": null,  
  "effectiveLevel": "INFO" }, "org.hibernate.SQL": { "configuredLevel": null, "effectiveLevel":  
  "INFO" } } }
```

GET /actuator/loggers/{name} — specific logger:

```
# GET /actuator/loggers/com.ecom.application { "configuredLevel": null, "effectiveLevel":  
"INFO" }
```

POST — Change log level at RUNTIME (no restart needed!):

```
# POST /actuator/loggers/com.ecom.application # Body: { "configuredLevel": "DEBUG" } # Now all  
your app classes log at DEBUG level – see full SQL, method calls etc. # To reset back to INFO:  
{ "configuredLevel": "INFO" }
```

Level	Shows	Use In
TRACE	Everything including internal framework calls	Never in prod — too noisy
DEBUG	Detailed app flow, SQL queries, method entry/exit	Dev or prod debugging
INFO	Normal business events, startup messages	Production (default)
WARN	Something unexpected but app still works	Production
ERROR	Failures, exceptions	Production
OFF	No logging at all	Rarely — specific noisy classes

When to use /loggers:

- Production issue — temporarily enable DEBUG to see detailed logs
- SQL not visible — set org.hibernate.SQL to DEBUG at runtime
- Turn off noisy library logging without touching properties file
- Protect with auth in production — anyone can enable DEBUG and flood logs

■ Interview Q: How do you debug a production issue without restarting?

Use /actuator/loggers to change log level to DEBUG at runtime. This shows detailed SQL, method calls, and flow without any restart. Once debugging is done, change back to INFO. This is only possible because of Actuator — without it you'd need a restart.

Securing Actuator Endpoints

This is the most important production concern with Actuator. Never expose all endpoints publicly. Use Spring Security to restrict sensitive endpoints to ADMIN role only.

Recommended production config:

```
# application-prod.properties # Only expose safe endpoints publicly
management.endpoints.web.exposure.include=health,info # Show minimal health details publicly
management.endpoint.health.show-details=when-authorized # Expose all to internal monitoring
(different port) management.server.port=9090 # actuator on different port, not public
```

Spring Security config for Actuator:

```
@Configuration public class SecurityConfig { @Bean public SecurityFilterChain
filterChain(HttpSecurity http) throws Exception { http.authorizeHttpRequests(auth -> auth //
Public endpoints .requestMatchers("/actuator/health").permitAll()
.requestMatchers("/actuator/info").permitAll() // Admin only – sensitive endpoints
.requestMatchers("/actuator/**").hasRole("ADMIN") // Your API endpoints
.requestMatchers("/api/**").authenticated() .anyRequest().authenticated() ); return
http.build(); } }
```

Endpoint	Dev	Staging	Production
/actuator/health	Public ■	Public ■	Public ■
/actuator/info	Public ■	Public ■	Public ■
/actuator/metrics	Public ■	ADMIN only	ADMIN only / Prometheus only
/actuator/loggers	Public ■	ADMIN only	ADMIN only
/actuator/env	Public ■	ADMIN only	Exclude completely ■
/actuator/beans	Public ■	ADMIN only	Exclude completely ■
/actuator/heapdump	Public ■	ADMIN only	Exclude completely ■
/actuator/shutdown	Disabled	Disabled	Disabled ■

■ Interview Q: How do you secure Actuator in production?

1. Only expose health and info publicly
2. Put sensitive endpoints behind ADMIN role with Spring Security
3. Run Actuator on a separate internal port (management.server.port)
4. Completely exclude env, beans, heapdump from production
5. Use show-details=when-authorized for health details

Actuator + Prometheus + Grafana Integration

This is the full production observability stack. Actuator exposes metrics, Prometheus collects them, Grafana visualizes them.

Step 1 — Add Micrometer Prometheus dependency:

```
<dependency> <groupId>io.micrometer</groupId>  
<artifactId>micrometer-registry-prometheus</artifactId> </dependency>
```

Step 2 — Expose prometheus endpoint:

```
management.endpoints.web.exposure.include=health,info,prometheus,metrics # Now  
/actuator/prometheus exposes metrics in Prometheus format
```

Step 3 — Full flow:

Spring Boot App ■■■ /actuator/prometheus (Micrometer exposes metrics) ↓ scrapes every 15s
Prometheus ↓ stores time-series data Grafana ↓ queries Prometheus Dashboards + Alerts ↓ alert
if CPU > 80%, DB connections full, error rate spikes PagerDuty / Slack notification

docker-compose.yml to run all together:

```
version: '3.8' services: app: image: ecom-app:latest ports: ["8080:8080"] prometheus: image:  
prom/prometheus ports: ["9090:9090"] volumes: -  
./prometheus.yml:/etc/prometheus/prometheus.yml grafana: image: grafana/grafana ports:  
["3000:3000"] depends_on: [prometheus]
```

■ This full stack (Actuator + Prometheus + Grafana) is what interviewers mean when they ask about 'observability' or 'monitoring' in microservices. Know this flow cold.

■ Notes built progressively alongside eCommerce Microservices Project. Next sections: SLF4J Logging, Docker, Kafka, Resilience4J, Spring Security.